Can Local LLMs Classify Political Texts? A Small Benchmark for Applied Researchers

10 political-science classification tasks, 6 models, 30,000 predictions

Hanno Hilbig

2026-04-27

Abstract

I benchmark four local open-weight models against two OpenAI API models on ten political-science classification tasks. The goal is not a definitive model ranking, but practical guidance for applied researchers: when are local models competitive, how much slower are they, and when does batching change the speed–accuracy tradeoff? Across tasks, the best local model is essentially tied with the strongest API model on average, but task-level uncertainty is substantial. Local models are slower on the largest dense architectures, but not uniformly so: the local mixture-of-experts model is the fastest model in the grid. Batching gives useful speedups on short-prompt tasks with limited accuracy loss at moderate batch sizes, but becomes fragile for long-codebook tasks, especially because of parse failures.

Bottom line. The benchmark suggests that local open-weight models are already competitive for many applied political-science classification tasks, but the benchmark does not support a simple local-versus-API ranking. The main practical differences come from task family, metric choice, and batching strategy.

1 Motivation

Applied political scientists who use LLMs for text classification face a recurring choice. The OpenAI and Anthropic APIs are reliable, well-documented, and easy to call from R or Python, but they cost money per item, send the data off-machine, and create reproducibility risk when the underlying model changes between versions. Local open-weight models running through Ollama or vLLM are free at the margin, keep the data on the researcher's machine, and freeze the version forever, but they are sometimes slower and sometimes worse on the specific task at hand.

The applied question is not "are local models as good as API models in general?" The applied question is "is the local model good enough on the specific task I want to run?" In this benchmark, the largest practical differences come from task family, metric choice, and inference setup. Aggregate model rankings are less informative than a small validation sample on the target task. The remainder of the report walks through how the answer varies across those axes, and where local-versus-API differences appear largest.

2 Design

The grid is ten classification tasks drawn from published political-science replication archives, six models (four local, two OpenAI), and N=500 items per task sampled with a fixed random seed. Headline metrics are macro F1 and, for class-imbalanced tasks, accuracy and Matthews correlation coefficient (MCC). All comparisons use 95% paired-by-item bootstrap confidence intervals (1000 iterations). Local inference runs on a single Apple Silicon MacBook (32 GB) through Ollama with thinking off; OpenAI calls go through the standard chat completions API with `reasoning_effort=medium` and a strict JSON schema. Some prompts

are verbatim from the source paper's coding instructions; others are derived from the source paper's codebook because the original study did not use an LLM. Per-task provenance is in `docs/prompts_provenance.md`.

Table 1: Benchmark scope at a glance.

Category	Included
Tasks	10 political-science classification tasks
Items	500 per task (random seed 20260422)
Predictions	30,000 serial + 44,500 batched
Local models	Gemma 4 31B, Qwen3 14B, Qwen3 30B-A3B (MoE), Mistral Small 24B (4-bit Q4_K_M)
API models	gpt-5.5, gpt-5.4-nano (both reasoning_effort=medium)
Metrics	macro F1, accuracy, MCC, latency, parse-error rate
Hardware	Apple M2 Pro, 32 GB unified memory, macOS Tahoe 26.1, Ollama

Per-task descriptions (object of classification, label set, family, provenance, prompt length) are listed in the Appendix.

3 Results

3.1 Performance: local models are competitive, but task variance dominates

The strongest API model and the strongest local model are essentially tied on average. gpt-5.5 has the highest mean macro F1 (0.621), but its lead over Gemma 4 31B (0.619) is 0.002 points. The top five models are within five F1 points of each other (Figure 1), while per-task spread within any single model exceeds that range. At the task level, most pairwise model differences fall inside bootstrap CI overlap; the benchmark therefore does not support a sharp global ranking. Among the two API models, gpt-5.4-nano is the more cost-efficient option in this grid: at $0.65 to $1.14 per 1000 correct predictions, it is about nine times cheaper than gpt-5.5, with a modest mean-F1 lag.

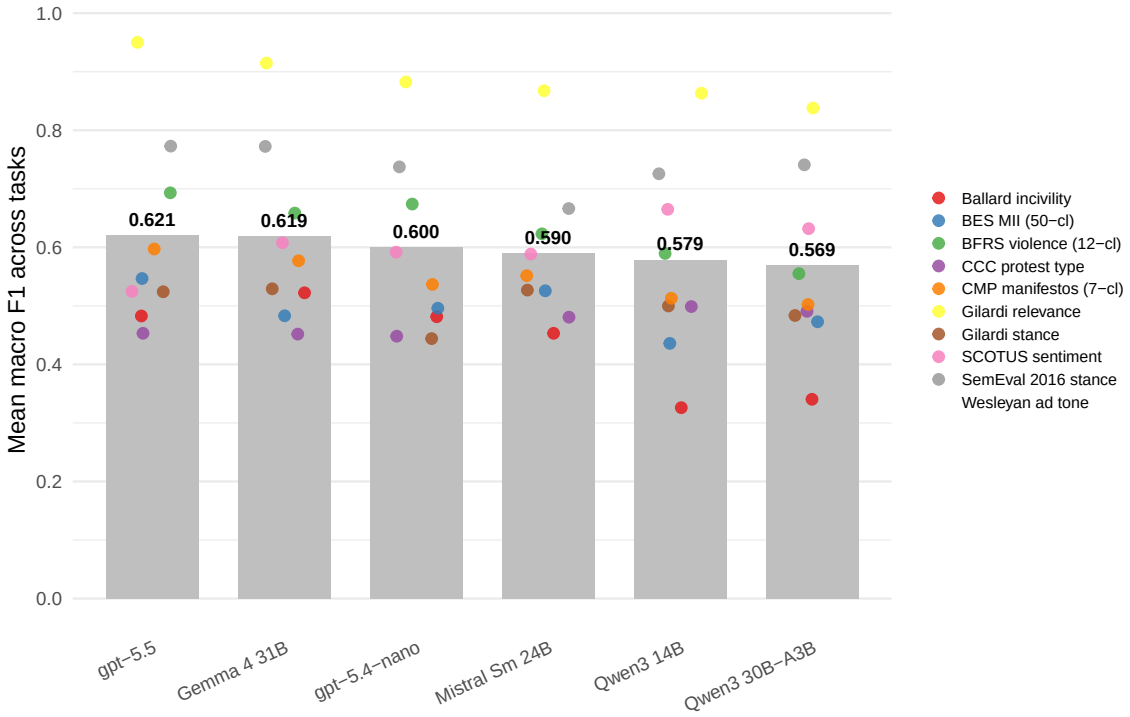

Figure 1: Mean headline F1 across ten tasks per model. Bars show cross-task means; coloured dots show per-task F1 values jittered horizontally for visibility.

The bigger lesson is task heterogeneity. Bootstrap CIs at N=500 are wide enough that on nine of ten tasks five or six models have confidence intervals that overlap with the top point estimate; only Gilardi relevance separates a clear top group of two. For applied researchers this means a small task-specific validation matters more than the choice between any two highly ranked models in this grid.

3.2 Task families: no universal winner

Different model classes lead on different families of tasks (Figure 2). gpt-5.5 narrowly leads on policy-topic coding (CMP manifestos, BES MII) and on relevance/incivility. Gemma 4 31B and the Qwen3 models trade leads across stance, sentiment, tone, and event-coding tasks. The result rules out a simple global recommendation. A small task-specific pilot is more informative than applying the aggregate leaderboard to a new coding problem.

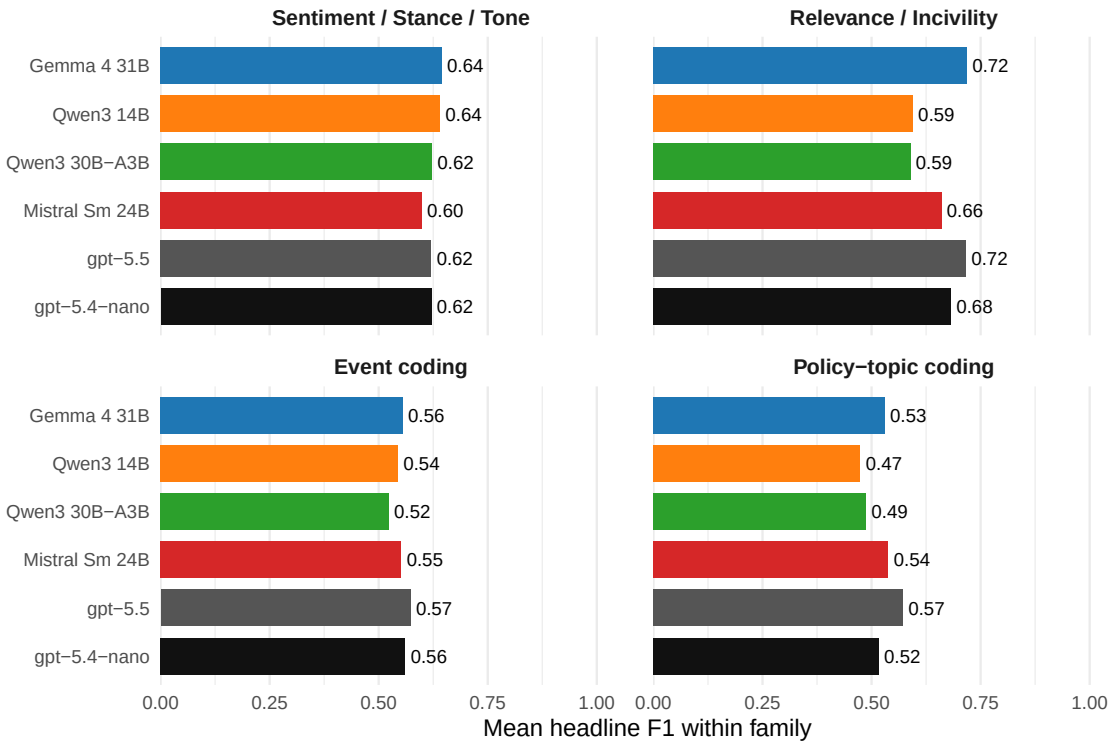

Figure 2: Mean headline F1 within task family, by model. Each family contains two to four tasks; family-level averages are descriptive only.

3.3 Speed: local inference is not uniformly slower

Speed in this grid is not the simple "API fast, local slow" story (Figure 3). The fastest model is local: Qwen3-30B-A3B (a mixture-of-experts architecture with 3B active parameters) has the lowest median per-item latency in the grid and is faster than either OpenAI tier. The slowest model is also local: Gemma 4 31B, the largest dense local model, has the highest median per-item latency in the grid. The two OpenAI tiers fall in the middle. The applied takeaway is that "local versus API" is not enough for thinking about speed; model architecture, quantization, and hardware constraints matter substantially.

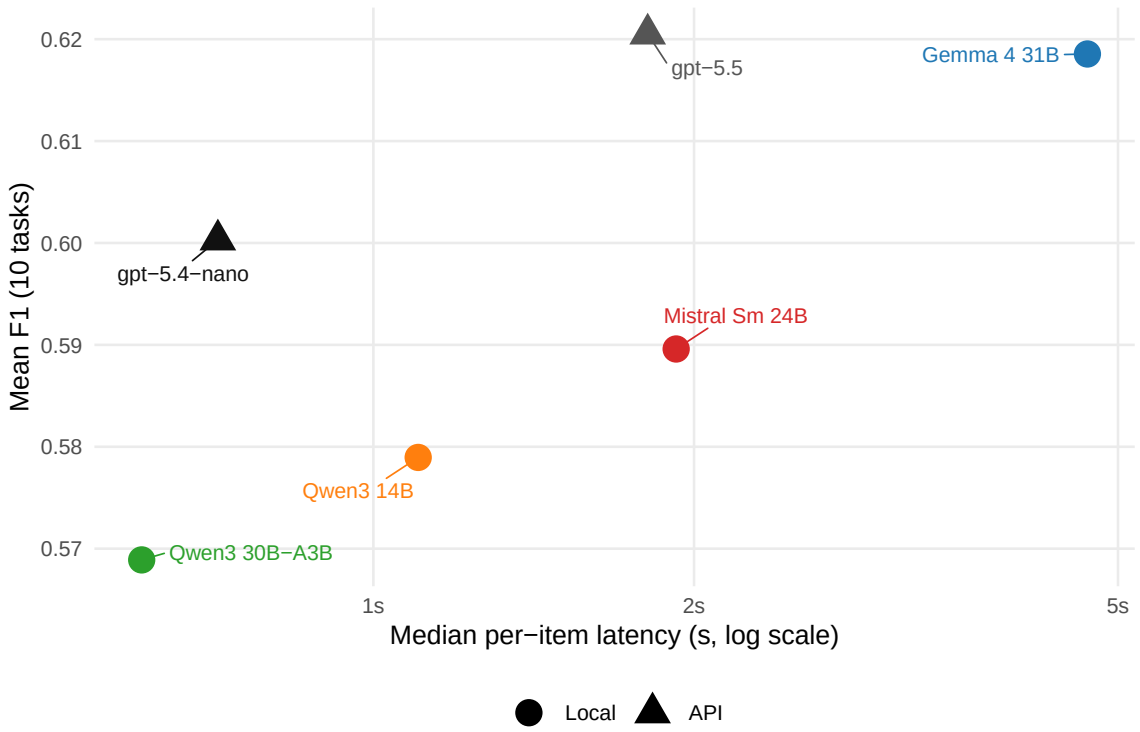

Figure 3: Mean F1 across tasks vs median per-item latency, per model. One labelled point per model; x-axis on log scale. Triangles mark API models.

Two points are worth keeping in mind. First, latencies on Apple Silicon depend strongly on which model fits in unified memory; a different machine will reorder these points, especially among the local models. Second, the OpenAI per-item latency includes network and serving overhead that batching can amortize (next subsection); the per-call wall-clock is more relevant for high-throughput workloads.

3.4 Batching: useful when feasible, fragile on long codebooks

Batched prompting (packing multiple items into one API call and asking for an array of classifications back) is the standard production speedup for API models (Pipal et al. 2026). This benchmark tested batched inference at b=10 and b=20 against a serial b=1 baseline. The headline picture (Figure 4) is that batching delivers useful speedups on short-prompt tasks with little or no accuracy loss, but becomes fragile on long-codebook categorical tasks where the system-prompt prefill is already long.

On short-prompt tasks (Gilardi relevance/stance, Ballard incivility, SemEval, SCOTUS, Wesleyan ad tone), all six models reach 1.5x to 4x per-item speedup at b=10 with mean-F1 changes inside the +/-0.03 band. b=10 captures most of the speedup; the move from b=10 to b=20 buys little extra throughput while raising fragility on smaller local models. On long-codebook tasks (Halterman CCC, BFRS, BES MII, CMP manifestos), two patterns emerge. Local Ollama batching does not help: the system prompt is already long, so packing additional items past the prefill barrier does not amortize anything. A pilot cell (Halterman CCC x Gemma x b=10) ran at 14.9 s/item, slightly slower than the serial 13.9 s/item baseline. OpenAI batching looks fast in latency terms, but parse-error rates climb sharply. Figure 4 does not fully capture this fragility because F1 is computed only on successfully parsed outputs; parse failures enter as a separate reliability problem. gpt-5.5 batched on Halterman CCC at b=10 emits malformed JSON arrays for 68% of items; gpt-5.5 on Wesleyan at b=20 hits 48%. gpt-5.4-nano on the same long-codebook tasks tops out at 12% parse errors and is the more reliable batched option for that workload.

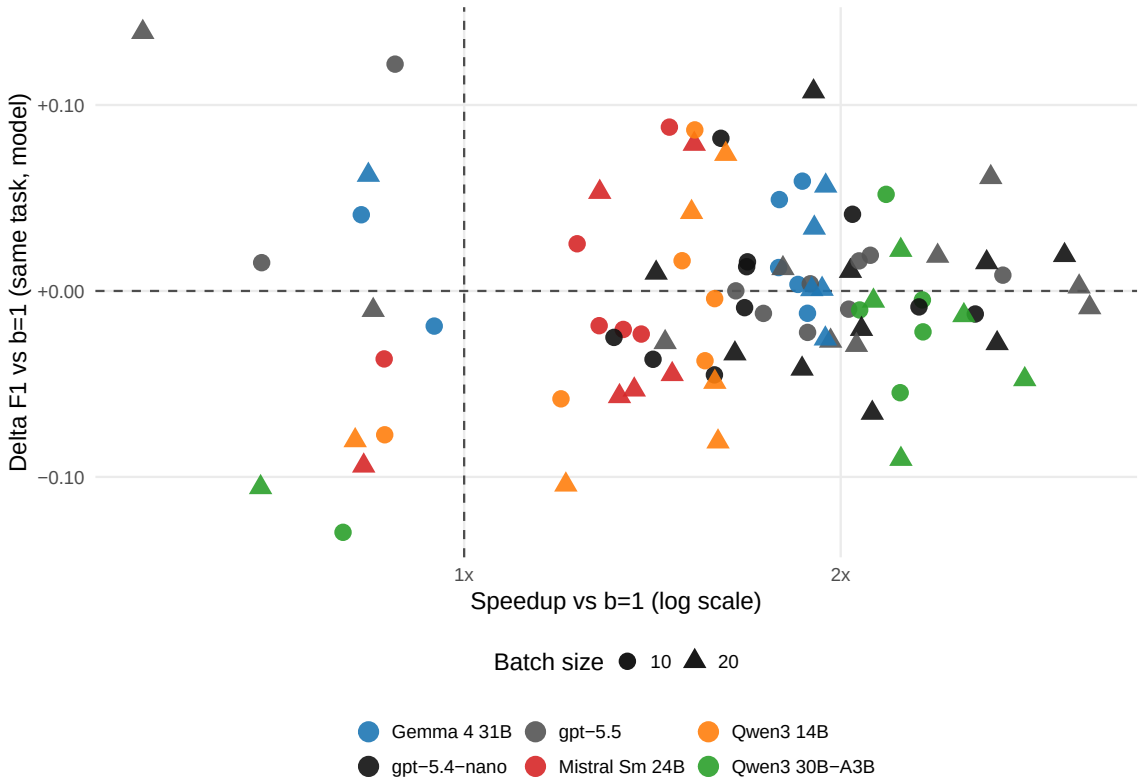

Figure 4: Per-cell speedup vs accuracy change relative to b=1, for batched cells with b=10 (circles) or b=20 (triangles). Each point is one (task, model, batch size) cell. Upper-right = faster and at-least-as-accurate; lower-right = fast but worse; left of x=1 = batching does not help here.

Interpretation and limits

Cost. The v2 OpenAI runs cost $22.86 in total for 10,000 API predictions: about $1.20 for gpt-5.4-nano and $21.66 for gpt-5.5. In this grid, gpt-5.4-nano is the cost-efficient API option, substantially cheaper than gpt-5.5 with only a modest mean-F1 loss. Local models are free at the margin but not costless; they trade API charges for wall-clock time, hardware constraints, electricity, and local setup overhead.

Hardware. All local inference ran through Ollama on an Apple M2 Pro machine with 32 GB unified memory under macOS Tahoe 26.1. The accuracy results are likely more portable than the throughput results. Latency can change substantially across Apple Silicon generations, memory configurations, NVIDIA GPUs, CPU-only machines, and quantization choices.

Model selection and prompting. The model set is practical rather than exhaustive: four local models that fit a 32 GB Apple Silicon setup, plus two OpenAI tiers that bracket a cheap and a flagship option. Prompts were not optimized separately per model; some are verbatim from the source paper, others are derived from published codebooks. This brings the exercise closer to typical applied use, but it is not a maximum-performance comparison; rankings could shift under heavier per-model prompt engineering.

Task averaging and rare labels. The mean F1 leaderboard is a descriptive summary, not an estimand. Tasks differ in label count, class imbalance, construct complexity, and prompt provenance. For imbalanced codebooks, average metrics can hide failures on rare classes; researchers using LLMs for production coding should inspect confusion matrices and class-specific recall, not only macro F1, accuracy, or MCC.

Privacy and restricted data. Local inference avoids transmission of text to a third-party API and can simplify workflows with sensitive or restricted data. It does not automatically resolve IRB, consent, copyright, data-use agreement, or security requirements. Researchers still need to verify whether local model use is

allowed under the relevant data agreement.

Fine-tuning and model drift. This benchmark studies prompt-based classification. It does not compare against fine-tuned local models, supervised classifiers, or embedding-based models; for large labeled datasets and stable codebooks, those alternatives may be cheaper and more reliable. API endpoints can also change behavior between releases even when the model name is stable, while local open-weight models are easier to freeze exactly. Re-evaluate when models change.

Practical guidance

- Run a small task-specific validation sample before production use; the benchmark mean does not predict the per-task outcome reliably at this scale.
- For imbalanced label sets, report accuracy and MCC alongside macro F1. Mellon BES MII is a clean illustration: macro F1 of about 0.45 looks like a hard task, but accuracy is 0.94 and MCC is 0.93.
- For short-prompt tasks, b=10 is a reasonable starting point. The accuracy cost is small and the speedup is real.
- For long-codebook categorical tasks, do not batch aggressively unless retries are built in. Schema violations on gpt-5.5 hit 30 to 68 percent at b=10/20 on Halterman CCC, BFRS, and Wesleyan.
- A model that wins one political-text task should not be treated as the default winner for another. In this grid, task-level reversals are common.

Caveats

- The model set is convenience-selected and the task set is heterogeneous; the leaderboard is descriptive, not authoritative.
- N=500 still leaves wide uncertainty for many task-level comparisons. On nine of ten tasks the bootstrap CIs put five or six models within CI overlap of the top point estimate.
- Macro F1 is misleading as the only headline metric for imbalanced tasks like BES MII (50 classes, dominated by one), Wesleyan ad tone, and Ballard incivility. Accuracy and MCC are reported alongside.
- Hardware-specific. All local inference ran on Apple Silicon with 32 GB unified memory through Ollama. Throughput on other hardware will reorder the latency results.
- Snapshot, not stable: gpt-5.5 launched two days before the rerun and changed the leaderboard relative to v1. Re-evaluate when models change.

References

Ballard, A. (2022). Civility in Congressional speech. *Legislative Studies Quarterly*.

Chae, Y., & Davidson, T. (2025). Large language models for text classification: From zero-shot learning to fine-tuning. *Sociological Methods & Research*.

Gilardi, F., Alizadeh, M., & Kubli, M. (2023). ChatGPT outperforms crowd workers for text-annotation tasks. *Proceedings of the National Academy of Sciences* 120(30).

Halterman, A., & Keith, K. A. (2025). Codebook LLMs: Evaluating language models on classification tasks defined by long, multi-class codebooks. *Political Analysis*.

Mellon, J., Bailey, J., Scott, R., Breckwoldt, J., & Miori, M. (2024). Do AIs know what the most important issue is? Using language models to code open-text social survey responses at scale. *Research & Politics*.

Ornstein, J. T., Blasingame, E. N., & Truscott, J. (2025). How to train your stochastic parrot: Open-source large language models for political-science text classification. *Political Science Research and Methods*.

Pipal, M., Vogel, S. J., Wack, M., & Esser, F. (2026). Batched prompting for fast and reproducible large-scale text annotation. *arXiv:2604.03684*.

Zhang, X., Fowler, E. F., Franz, M. M., & Ridout, T. N. (2025). Wesleyan creative ads coding (2022 cycle). *Scientific Data*.

Appendix link

Full per-task tables, parse-error tables, prompt provenance, batched-inference details, summary CSVs, and reproduction instructions are in the HTML companion at `output/report.html` (and on the GitHub repository).

Appendix: Tasks

Table 2: Tasks in the benchmark, with traits referenced in the main text. 'Family' is the grouping used in Section 3.2. 'Coded object' is the substantive item the model classifies. 'Provenance': Verbatim = prompt unchanged from source paper; Verbatim-adapted = source content with format-only changes (single-item, JSON tail, label subset); Derived = our prompt reasoned from the published codebook. 'Prompt': short (~15-35 lines) or long codebook (~47-126 lines); this is the trait that determines local-batching feasibility in Section 3.4.

Task	Source	Family	Coded object	Labels	Provenance	Prompt
Gilardi relevance	Gilardi 2023	Relevance / Incivility	Tweets, classified as about content moderation or not	binary: relevant / not	Verbatim	short
Ballard incivility	Ballard 2022	Relevance / Incivility	Tweets by U.S. members of Congress, pre-filtered as divisive	binary: uncivil / not	Derived	short
Gilardi stance	Gilardi 2023	Sentiment / Stance / Tone	Tweets about content moderation	3-class: pro / neutral / contra	Verbatim	short
SemEval stance	Chae & Davidson 2025	Sentiment / Stance / Tone	Tweets toward a named political target (Trump, abortion, etc.)	3-class: FAVOR / AGAINST / NONE	Derived	short
SCOTUS sentiment	Ornstein et al. 2025	Sentiment / Stance / Tone	Tweets about a U.S. Supreme Court ruling	3-class: positive / negative / neutral	Derived	short
Wesleyan ad tone	Zhang et al. 2025	Sentiment / Stance / Tone	Political ads on Meta (Facebook / Instagram) toward a candidate	3-class: promote / attack / contrast	Derived	short
CCC protest type	Halterman & Keith 2025	Event coding	News stories about U.S. protest events	4-class: PROTEST / RALLY / DEMONSTRATION / MARCH	Verbatim-adapted	long codebook
BFRS violence	Halterman & Keith 2025	Event coding	News stories about Pakistani political violence	12-class: assassination, terrorism, riot, …	Verbatim-adapted	long codebook
CMP manifestos	Halterman & Keith 2025	Policy-topic coding	Quasi-sentences from political party manifestos (CMP/MARPOR)	7-class: economy, welfare, external relations, …	Derived	long codebook
BES MII	Mellon et al. 2024	Policy-topic coding	Open-ended British Election Study responses (most-important-issue)	50-class, imbalanced: Coronavirus is ~38% of items	Verbatim-adapted	long codebook